

GreenBite — Informe de Persistencia de Datos

Desarrollo Fullstack III · Evaluación Parcial N°3

1. Estrategia de persistencia: Database per Service

GreenBite aplica el patrón **Database per Service**: cada microservicio posee su propia base de datos **PostgreSQL**, aislada del resto. Ningún servicio accede a la base de datos de otro; la comunicación es siempre vía API REST a través del BFF. Esto garantiza autonomía, aislamiento de fallas y evolución independiente del esquema.

Microservicio	Base de datos	Motor	Puerto
ms-usuarios	usuarios_db	PostgreSQL 16	5433
ms-pedidos	pedidos_db	PostgreSQL 16	5434

Ambas instancias se levantan con Docker Compose, cada una con su propio volumen persistente (usuarios_data y pedidos_data), de modo que los datos sobreviven al reinicio de los contenedores.

2. Tecnología: JPA / Hibernate (Spring Data JPA)

La persistencia se implementa con **JPA (Jakarta Persistence API)**, usando **Hibernate** como proveedor a través de **Spring Data JPA**. JPA es un estándar de mapeo objeto-relacional (ORM): las clases Java anotadas con `@Entity` se mapean a tablas, y sus atributos a columnas. No se escribe SQL manual: Hibernate genera y ejecuta las sentencias, y los repositorios derivan las consultas a partir del nombre de sus métodos.

3. Configuración de la conexión

Cada microservicio define su datasource en `src/main/resources/application.properties`.

Ejemplo de `ms-usuarios`:

```
server.port=4001
spring.datasource.url=jdbc:postgresql://localhost:5433/usuarios_db
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
```

El pool de conexiones (**HikariCP**) lo autoconfigura Spring Boot. La propiedad `ddl-auto=update` hace que Hibernate cree o actualice las tablas automáticamente al arrancar, a partir de las entidades.

4. Modelo de datos (entidades)

Entidad **Usuario** → tabla `usuarios`:

Columna	Tipo	Restricción
id	UUID	PK (generado en <code>@PrePersist</code>)
nombre	TEXT	NOT NULL
email	TEXT	NOT NULL, UNIQUE
password_hash	TEXT	NOT NULL (BCrypt)
created_at	TIMESTAMP	NOT NULL

Entidad **Pedido** → tabla `pedidos`:

Columna	Tipo	Restricción
id	UUID	PK (generado en <code>@PrePersist</code>)
user_id	UUID	NOT NULL

plan	INT	NOT NULL (1, 2 o 4)
status	TEXT	NOT NULL (ej. PENDIENTE)
total	INT	NOT NULL (calculado por plan)
created_at	TIMESTAMP	NOT NULL

Ejemplo del manejo con anotaciones JPA:

```
@Entity @Table(name = "usuarios")
public class Usuario {
    @Id @Column(name="id") private UUID id;
    @Column(nullable=false) private String nombre;
    @Column(unique=true, nullable=false) private String email;
    @Column(name="password_hash") private String passwordHash;
    @Column(name="created_at") private Instant createdAt;
    @PrePersist void onCreate() { if (id==null) id=UUID.randomUUID(); ... }
}
```

5. Acceso a datos: Repository Pattern con Spring Data JPA

Los repositorios son interfaces que extienden `JpaRepository`. Spring genera la implementación en tiempo de ejecución, incluyendo operaciones CRUD (save, findById, findAll, deleteById) y las *query methods* derivadas del nombre:

```
public interface UsuarioRepository extends JpaRepository<Usuario, UUID> {
    Optional<Usuario> findByEmail(String email);
    boolean existsByEmail(String email);
    List<Usuario> findAllOrderByCreatedAtDesc();
}
```

6. Garantías de integridad y persistencia

- **Transaccionalidad:** las operaciones de escritura de Spring Data JPA (save, deleteById) se ejecutan dentro de una transacción gestionada por Spring, asegurando atomicidad.
- **Integridad:** restricciones a nivel de columna (NOT NULL, UNIQUE en email) impiden datos inconsistentes; el email duplicado se rechaza con HTTP 409.
- **Identificadores:** cada registro usa un UUID generado por la aplicación antes de persistir (@PrePersist), evitando colisiones entre servicios.
- **Aislamiento por servicio:** al tener bases separadas, un fallo en pedidos_db no afecta a usuarios_db.

7. Evidencia de persistencia

Tras ejecutar el flujo completo (registro de usuarios y creación de pedidos a través del BFF), se verificó que Hibernate creó las tablas y que los datos quedaron almacenados de forma persistente en PostgreSQL:

```
$ docker compose exec usuarios_db psql -U postgres -d usuarios_db -c "\dt"
public | usuarios | table | postgres

$ docker compose exec pedidos_db psql -U postgres -d pedidos_db -c "\dt"
public | pedidos | table | postgres
```

8. Conclusión

La persistencia de GreenBite combina el patrón arquitectónico **Database per Service** con el patrón de diseño **Repository** implementado sobre **JPA/Hibernate**. Esto desacopla la lógica de negocio del acceso a datos, centraliza el mapeo objeto-relacional en las entidades y garantiza integridad, transaccionalidad y autonomía de cada microservicio.